ALTINBAŞ UNIVERSITY

Faculty of Engineering and Architecture
Software Engineering Department

GRADUATION PROJECT REPORT

Pharmacy Management System

Student: Deya Ghanem
Student Number: 220513098
Academic Year: 2026-2027

Report status: Cumulative report for the project stage documented to date. It consolidates the project definition, requirements baseline, and Week 3 architecture design. Planned and unverified work is identified accordingly.

Document Status and Scope

This is the initial cumulative Graduation Project Report, assembled from the project README and the available Week 3 design report. It is intended to evolve as the project progresses. It does not claim that the full application has been implemented, tested, or deployed.

1. Introduction

The Pharmacy Management System is planned as a Windows desktop application intended to help pharmacies organize routine operations through one system. The project focuses on medicine search and information management, sales, and inventory control. The broader project vision also mentions purchases, suppliers, customers or patients, user management, and reports; these broader functions require further requirements clarification and may be implemented in later stages.

1.1 Problem Statement

Pharmacy staff need to locate medicines, check availability, complete sales, and maintain stock information. The project aims to bring these operations together in an organized interface. The specific current-state problems, workflows, and baseline performance have not yet been measured, so this report does not claim quantified improvements.

1.2 Project Objective

The objective is to develop a fast, organized, and easy-to-use pharmacy management system that supports medicine search, sales, and inventory management, with a design that can be extended as requirements are confirmed.

1.3 Target Users

The intended users are pharmacists, pharmacy employees, and administrators. Different permissions are planned according to responsibilities; the detailed role-permission matrix remains to be specified and tested.

2. Proposed System

The initial proposed system is a Windows desktop application for one pharmacy computer. Its initial functional focus is searching medicines by name or barcode, viewing medicine details, adding items to a sale, calculating totals, saving sales, and updating stock. Low-stock alerts, expiry alerts, and basic reports are planned features whose detailed acceptance criteria remain to be finalized.

2.1 Initial Scope

In scope for the initial design: Windows desktop UI; medicine search and information management; sales and inventory management; and a foundation for basic stock alerts and reports. Out of scope for the initial version: AI-based demand forecasting, online pharmacy/e-commerce functionality, external healthcare-system integrations, cross-platform support, cloud dependencies, and multi-branch synchronization.

2.2 Assumptions and Constraints

The current design assumes one Windows computer and local SQLite storage. The selected technologies are a proposed baseline, not an instructor-imposed constraint. Currency, tax, discount, supplier/purchase workflows, and detailed customer/patient data requirements have not been confirmed in the available materials and must be clarified before implementation.

3. Requirements Baseline

The Week 3 design report maps the following selected requirements to architecture and verification activities. The IDs below are retained from that design report; this table is not a replacement for the complete requirements specification.

ID	Requirement	Planned verification
FR-01	Search by medicine name or barcode.	Seed test records and verify matches and no-match behavior.

ID	Requirement	Planned verification
FR-03	Add sale items and calculate the total.	Test multiple items, quantity boundaries, and total calculations.
FR-04	Save a sale and update stock.	Test successful sale, insufficient stock, and forced database failure.
NFR-01	Search within 2 seconds with 5,000 records.	Seed 5,000 records and time ten searches; record actual results when tested.
NFR-02	Maintain sale and inventory consistency.	Test success, cancellation, insufficient stock, database error, and repeat confirmation.

These verification activities are planned; results should only be reported after the tests are actually run. Requirements outside this selected subset should be maintained in docs/requirements.md.

4. Methodology

The project is being developed incrementally. The current workflow uses weekly milestones, written requirements, architecture/design artifacts, GitHub issues, and version-control commits. Each milestone should produce reviewable evidence, such as updated documentation, diagrams, tests, or a working prototype. The Week 4 goal is a small end-to-end vertical slice rather than the complete application.

At each stage, requirements should be linked to design elements and verified with explicit tests. This report distinguishes planned behavior from implemented and verified behavior; implementation status must be updated as evidence becomes available.

5. System Architecture

The proposed architecture is a layered monolithic desktop application with four logical layers: presentation, application services, domain logic, and data access. SQLite is the local persistence store. The architecture aims to keep UI code separate from business rules and database operations.

5.1 Layer Responsibilities

Layer	Responsibility
Presentation (WPF)	Displays search, sales, and inventory screens; gathers user actions and shows results, totals, confirmations, and errors.
Application Services	Coordinates use cases, validation flow, and calls to domain and data-access components.
Domain Logic	Applies medicine, stock, and sale rules; validates quantities and calculates totals.
Data Access	Encapsulates SQLite queries, persistence, and transaction handling.
SQLite Database	Stores medicine, sale, sale-item, and user records locally.

5.2 Sale Data Flow

1. Staff searches for a medicine by name or barcode. 2. The UI sends the query to an application service, and the data-access layer queries SQLite. 3. Matching medicines and quantities are returned. 4. Staff selects a medicine and quantity; domain logic validates stock and calculates totals. 5. On confirmation, one transaction inserts the sale header and sale lines and decrements stock. 6. The UI displays success only after commit. If validation or a database error prevents completion, the operation fails and the transaction is rolled back where applicable; stock must remain unchanged after a failed sale.

5.3 Data Model (Preliminary)

Entity	Key fields	Rules / relationships
Medicine	MedicineId, Name, Barcode, Strength, Form, UnitPrice, QuantityInStock, ExpiryDate, IsActive	Name required; stock non-negative; barcode unique when present.
Sale	SaleId, CreatedAt, UserId, TotalAmount, Status	UserId identifies the staff member who recorded the sale.
SaleItem	SaleItemId, SaleId, MedicineId, Quantity, UnitPriceAtSale, LineTotal	Quantity > 0; preserves price at sale time; references Sale and Medicine.
User	UserId, Username, PasswordHash, Role, IsActive	Unique username; do not store plaintext passwords.

This is a preliminary entity sketch, not a final schema. Currency, tax, discount, and other business rules must be confirmed before implementation.

6. Technology Selection

C# with WPF on modern .NET is the proposed desktop technology baseline. WPF is Windows-focused and supports XAML, layout controls, and data binding, which fit the initial Windows desktop target. Alternatives considered in the design report include Windows Forms and Python/PySide6. The trade-off is that WPF does not provide a cross-platform UI.

SQLite is the proposed local database. It supports transactions and avoids requiring a separate database server for the single-computer prototype. PostgreSQL and SQL Server were identified as alternatives. If multi-branch operation or higher concurrent-write demands become requirements, the database decision should be reassessed.

Version control is Git and GitHub. Exact .NET version and package compatibility should be confirmed during implementation. These technology selections remain proposed decisions until validated in a working prototype.

7. Design Decisions

ADR-01 — C# + WPF on modern .NET. Decision: use WPF for the Windows-first desktop UI. Alternatives: Windows Forms or Python/PySide6. Rationale: fit with a multi-screen Windows desktop application and support for XAML and data binding. Trade-off: Windows-specific UI.

ADR-02 — SQLite for local persistence. Decision: use SQLite for the initial single-computer prototype. Alternatives: PostgreSQL or SQL Server. Rationale: local deployment and transaction support without a separate database server. Trade-off: reassess if centralized, multi-branch, or high-concurrency operation is later required.

8. Security, Failure Handling, and Risks

The design calls for authentication for privileged actions, role-based authorization for edits, validation of required fields and quantities, non-negative stock, password hashing rather than plaintext storage, transaction rollback on sale failures, and tested backup/restore procedures. These are design requirements, not claims of completed implementation.

Risk	Impact	Mitigation
Partial sale persistence or duplicate stock deduction	Incorrect sales records or inventory quantities.	Use one transaction; test forced failure, cancellation, insufficient stock, and repeated confirmation.
Slow or ambiguous medicine search	Checkout delay or selection of the wrong medicine.	Index suitable fields, validate barcode uniqueness where present, and test the 5,000-record / 2-second target.

9. Week 4 Prototype Goal and Acceptance Evidence

The next milestone is to demonstrate a Windows desktop screen that searches seeded medicine records by name or barcode, lets a user select a medicine and enter a quantity, confirms a test sale, and displays the saved sale with reduced stock. Evidence should include a screenshot of the UI and database output before and after the sale. If full integration takes longer, transaction behavior may first be demonstrated through a minimal UI or test harness.

10. Project Status and Next Steps

Documented to date: project definition and initial scope in README; a requirements baseline in docs/requirements.md; proposed architecture, selected requirement traceability, component responsibilities, sale flow, preliminary data model, two technology decisions, risks, and a Week 4 prototype goal in the Week 3 design artifacts.

Not claimed as complete: full application implementation, execution of the planned performance and transaction tests, deployment, or validation with real pharmacy workflows. These should be marked complete only after evidence is available.

Next steps: (1) confirm open business rules and detailed requirements; (2) implement the Week 4 vertical slice; (3) test search performance and sale rollback; (4) update this report with implementation and test evidence; (5) keep README, weekly progress logs, and GitHub commits aligned with actual progress.

References

1. Microsoft Learn, WPF overview: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/index
2. SQLite, SQLite Is Transactional: https://www.sqlite.org/transactional.html
3. Microsoft .NET, releases and support: https://learn.microsoft.com/en-us/dotnet/core/releases-and-support